

1. Összetett objektumok és egymásba ágyazott adatok

A JavaScriptben az objektumok kulcs-érték párokat tárolnak. Gyakran előfordul, hogy egy objektumon belül egy másik objektum vagy tömb is helyet kap (ez a nesting / egymásba ágyazás). Ez a szerkezet teszi lehetővé a komplex, valós életbeli adatok modellezését.

- **Példa 1:** Komplex felhasználói profil elérése biztonságosan optional chaining (?.) operátorral.

```
const userProfile = { id: 101, personalInfo: { firstName: "Anna", lastName: "Kovács" }
};
console.log(userProfile.personalInfo.firstName); // "Anna"
console.log(userProfile.address?.city); // undefined
```

2. Gyakorlás: Objektumok és nyílfüggvények

Az objektumokhoz metódusokat (függvényeket) is rendelhetünk. Rendkívül fontos megérteni a különbséget a hagyományos függvények és a nyílfüggvények (arrow functions) között a 'this' kulcsszó viselkedésében.

```
const car = {
  brand: "Toyota", model: "Corolla",
  getFullDescription: function() { return `${this.brand} ${this.model}`; },
  getArrowDescription: () => { return `${this.brand} ${this.model}`; }
};
console.log(car.getFullDescription()); // "Toyota Corolla"
console.log(car.getArrowDescription()); // "undefined undefined"
```

3. Tömbök alapjai és elemek elérése index szerint

A tömbök rendezett adatlisták, ahol az elemeket 0-tól kezdődő sorszám (index) alapján érjük el.

```
const fruits = ["Alma", "Banán", "Cseresznye"];
fruits[1] = "Kivi";
const queue = ["Első"]; queue.push("Második"); queue.shift();
```

4. Tömb bejárás: forEach vs .map() (JSX listák alapja)

A forEach mellékhatásokért felel, míg a .map() minden elemet átalakít és egy teljesen új tömbbel tér vissza.

```
const prices = [1000, 2000, 5000];
const pricesWithVAT = prices.map(price => price * 1.27);
```

5. Adatszűrés: .filter() (Keresés és kategóriák alapja)

A .filter() metódus egy új tömböt hoz létre mindazokból az elemekből, amelyekre a megadott tesztfüggvény értéke true.

```
const products = [{ id: 1, category: "tech" }, { id: 2, category: "fashion" }];  
const techProducts = products.filter(p => p.category === "tech");
```

6. Keresés tömbben: .find() és .findIndex() (ID keresés)

A .find() az első olyan elemet adja vissza, amely megfelel a feltételnek. A .findIndex() ugyanezen elem indexét adja vissza.

```
const users = [{ id: 101, name: "Anna" }];  
const user101 = users.find(u => u.id === 101);
```

7. Aggregálás: .reduce() (Kosár végösszeg kiszámítása)

A .reduce() metódus a tömb elemeit egyetlen kumulált értékévé fűzi össze a megadott akkumulátor függvény segítségével.

```
const cart = [{ price: 4000, qty: 2 }, { price: 12000, qty: 1 }];  
const grandTotal = cart.reduce((acc, item) => acc + (item.price * item.qty), 0);
```

8. Tömb destrukurálás és spread operátor ([...items])

A destrukurálás lehetővé teszi a tömbök elemeinek kényelmes kibontását változókba, míg a spread operátor (...) kiterjeszti a tömb elemeit.

```
const [x, y, z = 0] = [10, 20];  
const combined = [...["piros"], "fehér", ...["kék"]];
```

9. Gyakorlás: Láncolt tömbmetódusok (filter().map())

Mivel a .filter() és a .map() is új tömböt ad vissza, közvetlenül egymás után láncolhatóak a hatékony adatfeldolgozáshoz.

```
const inventory = [{ name: "TV", price: 200000, available: true }];  
const discountedPrices = inventory.filter(i => i.available).map(i => i.price * 0.8);
```

10. Eseménykezelés alapjai (addEventListener, DOM események)

A DOM (Document Object Model) események (events) teszik interaktívá a weboldalakat. Amikor a felhasználó kattint, gépel, görget vagy az oldal betöltődik, a böngésző eseményeket generál. Az 'addEventListener' a legmodernebb módja ezen események figyelésének, mivel egyazon elemhez több különböző (vagy akár azonos) eseményfigyelőt is csatolhatunk anélkül, hogy felülrírnánk egymást. Az eseménykezelő függvény (callback) paraméterként megkapja az Event objektumot (gyakran 'e' vagy 'event' néven), amely minden részletet tartalmaz a történekekről (pl. hol történt a kattintás, melyik gombot nyomták meg).

- **Példa 1:** Esemény delegálás (Event delegation) és az Event objektum használata.

```
// Tegyük fel, hogy van egy <ul> listánk sok <li> elemmel.
// Nem teszünk mindegyik <li>-re külön figyelőt, hanem a szülő <ul>-re tesszük
// (delegálás).
const list = document.getElementById("myList");

list.addEventListener("click", function(event) {
  // event.target: a legbelső elem, amire TÉNYLEGESEN rákattintottak (pl. egy <li>)
  // event.currentTarget: az az elem, amire a figyelő rá van kötve (itt a <ul>)

  if (event.target.tagName === "LI") {
    console.log("Kattintottál egy listaelemre:", event.target.textContent);

    // Stílus módosítása a kattintott elemen
    event.target.style.backgroundColor = "yellow";
  }
});
```

- **Példa 2:** Alapértelmezett viselkedés blokkolása és eseményfigyelő eltávolítása.

```
const form = document.getElementById("registerForm");
const submitBtn = document.getElementById("submitBtn");

// 1. preventDefault(): Megakadályozza a böngésző gyári viselkedését (pl. oldal
// újratöltése form küldéskor)
form.addEventListener("submit", (e) => {
  e.preventDefault();
  console.log("A form adatai feldolgozásra kerültek, de az oldal nem töltődött
  újra.");
});

// 2. removeEventListener(): Figyelő lekapcsolása. Csak névvel ellátott függvénnyel
// működik!
function onceClickHandler(e) {
  console.log("Ez a gomb csak egyszer volt megnyomható!");
  // Eltávolítjuk a figyelőt önmagán belül
```

```
submitBtn.removeEventListener("click", onceClickHandler);
}

submitBtn.addEventListener("click", onceClickHandler);
```

11. Aszinkron JavaScript: Event Loop és Callback-ek

A JavaScript egy 'single-threaded' (egyszálú) nyelv, vagyis egyszerre csak egyetlen dolgot tud végrehajtani a Call Stack-en (Hívási Verem). Hogy mégse fagyjon le a böngésző egy lassú hálózati kéréstől, az aszinkron feladatokat a böngésző Web API-jai (vagy Node.js esetén a C++ API-k) veszik át. Amikor ezek végeznek, a hozzájuk tartozó callback függvények egy várakozási sorba (Task Queue vagy Microtask Queue) kerülnek. Az Event Loop (Eseményhurok) folyamatosan figyeli a Call Stack-et, és ha az kiürül, betölti oda a sorban várakozó callbackeket. Fontos különbség: a Promise-ok a gyorsabb Microtask Queue-ba, míg a setTimeout a lassabb Macrotask Queue-ba kerül!

- **Példa 1:** Az Event Loop futási sorrendjének demonstrálása (Microtask vs. Macrotask).

```
console.log("1. Szinkron kód (Call Stack) - Azonnal lefut");

// Macrotask Queue (lassabb prioritás)
setTimeout(() => {
  console.log("4. setTimeout (Macrotask) - Csak a Microtaskok után fut le");
}, 0);

// Microtask Queue (magasabb prioritás)
Promise.resolve().then(() => {
  console.log("3. Promise (Microtask) - A szinkron kód után azonnal fut");
});

console.log("2. Szinkron kód (Call Stack) - Azonnal lefut");

// Konzol kimenet sorrendje: 1, 2, 3, 4
```

- **Példa 2:** A 'Callback Hell' (Callback pokol) - Miért kellettek a Promise-ok?

```
// Régen, a Promise-ok előtt mindent egymásba ágyazott callbackekkel oldottunk meg.
// Ez a "Pyramid of Doom", ami olvashatatlaná és nehezen karbantarthatóvá tette a kódot.

function downloadFile(url, callback) {
  console.log("Fájl letöltése...");
  setTimeout(() => callback("fajl_tartalma.txt"), 1000);
}

function processFile(file, callback) {
```

```

    console.log("Fájl feldolgozása...");
    setTimeout(() => callback("FELDOLGOZOTT_ADAT"), 1000);
}

function saveToDatabase(data, callback) {
    console.log("Mentés az adatbázisba...");
    setTimeout(() => callback("Sikeres mentés!"), 1000);
}

// Callback Hell a gyakorlatban:
downloadFile("http://example.com/file", (file) => {
    processFile(file, (processedData) => {
        saveToDatabase(processedData, (result) => {
            console.log("Végső eredmény:", result);
        });
    });
});
});

```

12. Promise-ok működése és állapotai

A Promise egy jövőbeli, aszinkron művelet eredményét (vagy hibáját) reprezentálja. Három állapota van:

1. Pending (folyamatban): az alapértelmezett állapot, amíg a feladat be nem fejeződik.
2. Fulfilled (teljesített): a művelet sikeresen véget ért (a 'resolve' hívódott meg).
3. Rejected (elutasított): valamilyen hiba történt (a 'reject' hívódott meg).

A Promise-ok `.then()`, `.catch()` és `.finally()` ágakkal láncolhatók (chaining), így elkerülhető a 'Callback pokol' (Callback hell).

- **Példa 1:** Saját Promise létrehozása, állapotok kezelése `.then()`, `.catch()` és `.finally()` metódusokkal.

```

const checkWeather = new Promise((resolve, reject) => {
    console.log("Időjárás lekérdezése folyamatban... (Pending)");

    setTimeout(() => {
        const isSunny = Math.random() > 0.5; // 50% esély szimulálása
        if (isSunny) {
            resolve("Süt a nap! (Fulfilled)"); // Átmenet Fulfilled állapotba
        } else {
            reject(new Error("Esik az eső... (Rejected)")); // Átmenet Rejected állapotba
        }
    }, 1500);
});

checkWeather
    .then(result => console.log("Siker:", result))

```

```

    .catch(error => console.error("Hiba:", error.message))
    .finally(() => console.log("Lekérdezés befejeződött (mindenképp lefut, akár siker, akár hiba)."));

```

- **Példa 2:** Több Promise kezelése egyszerre: `Promise.all()` és `Promise.allSettled()`.

```

const p1 = Promise.resolve("Adat 1");
const p2 = new Promise(resolve => setTimeout(() => resolve("Adat 2"), 1000));
const p3 = Promise.reject("Hiba a 3. kérésnél");

// Promise.all: Csak akkor sikeres, ha MIND sikeres (itt el fog hasalni a p3 miatt)
Promise.all([p1, p2])
    .then(results => console.log("Minden adat megjött:", results)) // ["Adat 1", "Adat 2"]
    .catch(err => console.error("Valamelyik elhasalt:", err));

// Promise.allSettled: Megvárja mindet, függetlenül attól, hogy sikeres vagy hibás
Promise.allSettled([p1, p2, p3]).then(results => {
    console.log("Minden kérés lezárult:");
    console.log(results[0].status); // "fulfilled"
    console.log(results[2].status); // "rejected"
});

```

13. async / await szintaxis használata

Az ES2017-ben bevezetett `async/await` szintaxis a Promise-ok kezelését teszi átláthatóbbá, olyanná, mintha szinkron kód lenne. Egy függvény elé írt `'async'` kulcsszó garantálja, hogy a függvény mindig egy Promise-szal tér vissza. Az `'await'` kulcsszó (amely csak `async` függvényen belül használható) megállítja a függvény futását, amíg az adott Promise fel nem oldódik, mindezt anélkül, hogy a böngésző főszálát blokkolná. A hibakezelés itt hagyományos `try...catch` blokkokkal történik.

- **Példa 1:** Aszinkron adatkérés és hibakezelés `try...catch...finally` segítségével.

```

async function processUserData() {
    console.log("Feldolgozás kezdete...");
    try {
        // A kódfutás itt "vár", amíg a fetch be nem fejeződik
        const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
        const data = await response.json();

        console.log("Feldolgozott adat:", data.name);
    } catch (error) {
        // Ha bármelyik await-elt Promise reject-el, a vezérlés ide ugrik
        console.error("Feldolgozási hiba:", error.message);
    } finally {
        // Ez a blokk a try és a catch után is lefut (pl. loading spinner eltüntetése)
    }
}

```

```

    console.log("Feldolgozás vége.");
  }
}

```

- **Példa 2:** Gyakori hiba: Szinkron várakozás vs. Párhuzamos végrehajtás.

```

async function slowExecution() {
  // LASSÚ: A második kérés csak az első befejezése után indul el (soros végrehajtás)!
  const data1 = await fetch("url1").then(r => r.json());
  const data2 = await fetch("url2").then(r => r.json());
}

async function fastExecution() {
  // GYORS: A kérések egyszerre indulnak el a háttérben...
  const req1 = fetch("url1").then(r => r.json());
  const req2 = fetch("url2").then(r => r.json());

  // ...majd egyszerre várjuk meg a befejezésüket Promise.all-al!
  const [data1, data2] = await Promise.all([req1, req2]);
  console.log("Mindkét adat egyszerre megérkezett:", data1, data2);
}

```

14. Adatlekérés: fetch() API hívások alapjai

A fetch() API a böngészők modern, beépített megoldása hálózati kérések (HTTP/HTTPS) indítására. Visszatérési értéke egy Promise, amely a szerver válaszát (Response stream) tartalmazza. Fontos sajátossága, hogy a fetch() csak hálózati hiba esetén dob kivételt (pl. nincs internet, megszakadt a kapcsolat). Ha a szerver 404 (Not Found) vagy 500 (Internal Error) hibakódot küld, a Promise sikeres (Resolved) marad! Ezért nekünk kell manuálisan ellenőrizni a 'response.ok' tulajdonságot (ami true, ha a HTTP státuszkód 200-299 között van).

- **Példa 1:** Robusztus GET kérés szigorú hibaellenőrzéssel és paraméterekkel.

```

async function fetchUserProfile(userId) {
  try {
    // URL paraméter dinamikus beillesztése Template Literál (``) segítségével
    const response = await
    fetch(`https://jsonplaceholder.typicode.com/users/${userId}`);

    // Manuális HTTP státuszkód ellenőrzés! (Ha 404-et kapunk, a fetch magától nem
    // dobna hibát)
    if (!response.ok) {
      throw new Error(`Szerver hiba! Státuszkód: ${response.status}`);
    }

    // A Response stream (pl. JSON formátum) JavaScript objektummá konvertálása

```

```

    const userData = await response.json();
    console.log(`Letöltött felhasználó: ${userData.name}, Email: ${userData.email}`);

    return userData; // Sikeres esetben ezzel térünk vissza

} catch (error) {
    console.error("Hálózati vagy szerver hiba történt:", error.message);
}
}

```

- **Példa 2:** POST kérés küldése JSON adatokkal és fejlécekkel (headers).

```

async function createNewArticle(articleData) {
    try {
        const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
            method: "POST", // HTTP metódus megadása (alapértelmezett a GET)
            headers: {
                // Megmondjuk a szervernek, hogy az elküldött tartalom JSON formátumú
                "Content-Type": "application/json; charset=UTF-8",
                // Példa autentikációra (API token)
                "Authorization": "Bearer my_secret_token_123"
            },
            // A JS objektumot szöveges JSON formátummá (string) kell alakítani küldés előtt
            body: JSON.stringify(articleData)
        });

        if (!response.ok) {
            throw new Error(`Mentés sikertelen! Kód: ${response.status}`);
        }

        const savedArticle = await response.json();
        console.log("Sikeresen létrehozva a szerveren, kapott ID:", savedArticle.id);

    } catch (error) {
        console.error("Hiba a mentés során:", error.message);
    }
}

// Függvény meghívása adatokkal
createNewArticle({
    title: "A Fetch API rejtelsei",
    body: "Ez a cikk a modern JS adatlekérésekről szól...",
    userId: 1
});

```